



Last updated on **Sep 11, 2026**

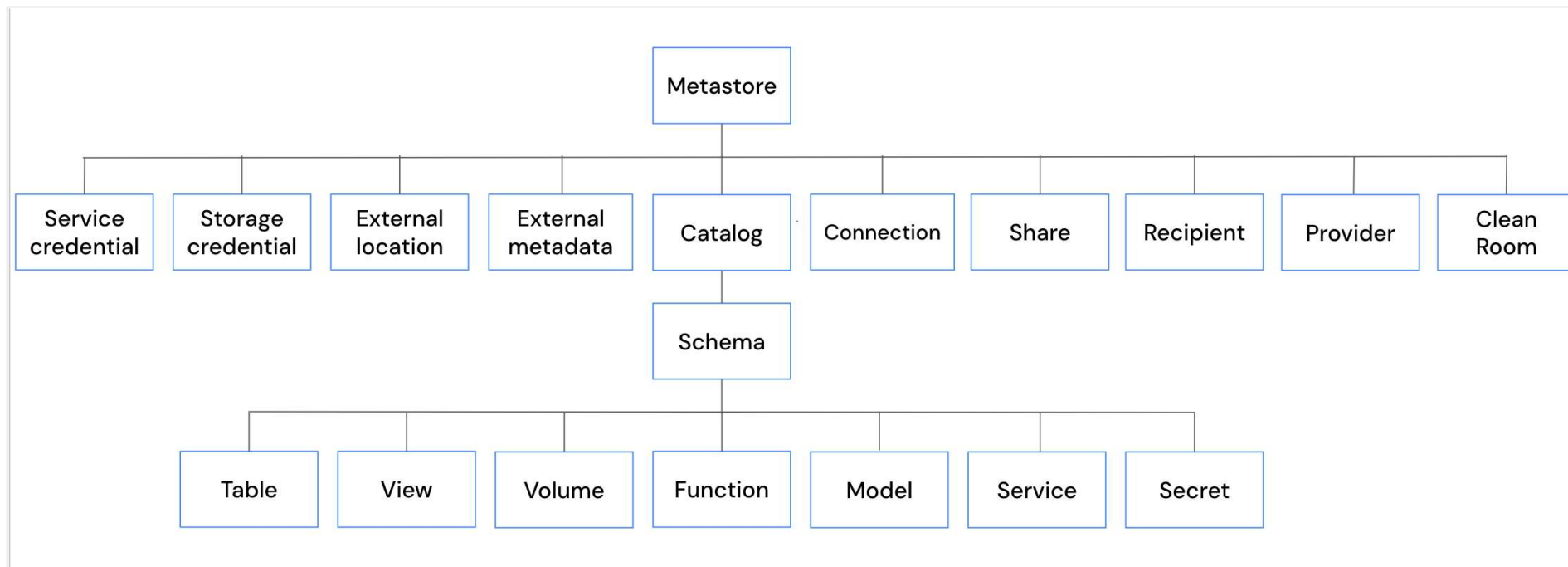
Unity Catalog permissions model concepts

This page explains core concepts of the Unity Catalog permissions model, including the object model, privileges, ownership, and inheritance.

For a general reference of all Unity Catalog privileges, see [Unity Catalog privileges reference](#). For instructions on granting and revoking privileges, see [Show, grant, and revoke privileges](#).

Securable objects

In Unity Catalog, data and metadata live in a top-level container called a metastore. Within this metastore, data is represented as objects in a three-level namespace: `catalog.schema.table`. This hierarchical structure also provides the foundation for access control in Unity Catalog.



Every object in this hierarchy is a **securable object**. Access control in Unity Catalog works by granting privileges, such as `SELECT`, `MODIFY`, or `USE SCHEMA`, on these securable objects. This model provides fine-grained control over who can access and manage data across your organization.

For a complete list of securable objects and the privileges that apply to each, see [Unity Catalog privileges reference](#).

Container objects

Some securable objects in Unity Catalog are **container objects**, meaning they contain child objects within the hierarchy. Container objects have a special role in the permissions model because privileges granted on them can affect their children through inheritance.

The following are container objects in Unity Catalog:

- **Catalogs:** The top level of the three-level namespace. Catalogs contain schemas as direct children.
- **Schemas:** The middle level of the three-level namespace. Schemas contain tables, views, volumes, and functions as direct children.

Container objects have several important characteristics:

Characteristic	Description
Privilege inheritance	When you grant a privilege on a container object, that privilege automatically applies to all current and future child objects. For example, granting <code>SELECT</code> on a catalog allows users to read all tables in that catalog (with appropriate usage privileges). See Privilege inheritance .
Usage privileges	Access to child objects requires the appropriate <code>USE CATALOG</code> or <code>USE SCHEMA</code> privilege on the parent container objects. See Usage privileges .
Ability to manage child objects	When you own a container object, you automatically get the ability to manage all child objects, even if you don't own those children directly. See Ownership .
Creation privileges	Container objects support privileges that allow users to create child objects within them, such as <code>CREATE SCHEMA</code> on catalogs and <code>CREATE TABLE</code> on schemas.

Non-container objects, like tables, views, volumes, and functions, don't contain child objects.

Privileges

Privileges determine what actions a user or group can perform on a securable object. Common privileges include:  **Ask Genie**

- `SELECT`: Read data from tables or views
- `MODIFY`: Write data to tables or views
- `USE CATALOG`: Access a catalog (requires additional privileges to work with child objects)
- `USE SCHEMA`: Access a schema (requires additional privileges to work with child objects)
- `CREATE TABLE`: Create tables within a schema

A user or group must be explicitly granted a privilege to perform an action.

The following sections describe important special privileges in Databricks. For a complete reference of all privileges, see [Unity Catalog privileges reference](#).

Usage privileges

`USE CATALOG` and `USE SCHEMA` are usage privileges. Generally, usage privileges are a prerequisite to interact with an object and its child objects in the hierarchy.

To work with any object in a catalog, you need the `USE CATALOG` privilege on the catalog, and to work with any object in a schema, you need the `USE SCHEMA` privilege on the schema. Managing an object with the `MANAGE` privilege is an exception, with reduced usage privilege requirements. See [Usage privilege requirements for `MANAGE`](#).

For example, to perform most operations on tables, views, volumes, or functions, you need:

1. `USE CATALOG` on the parent catalog
2. `USE SCHEMA` on the parent schema
3. The specific privilege for the operation (such as `SELECT`, `MODIFY`, or `EXECUTE`)

All three are required. Having only the `SELECT` privilege on a table is not sufficient to read it if you lack `USE CATALOG` or `USE SCHEMA` on its parent objects.



Usage privileges provide an important access control mechanism for higher-level administrators. For example, even if a table owner wants to share their table with other users, those users cannot access the table without `USE CATALOG` and `USE SCHEMA` privileges on the parent objects. Because only catalog and schema owners or users with the `MANAGE` privilege can grant these usage privileges, this prevents table owners from granting access outside approved boundaries.

Usage privilege requirements for `MANAGE`

The `MANAGE` privilege has reduced usage privilege requirements. To exercise `MANAGE`, you don't need a usage privilege at the level where `MANAGE` is granted: `MANAGE` on a catalog doesn't require `USE CATALOG` on that catalog, and `MANAGE` on a schema doesn't require `USE SCHEMA` on that schema. You need the usage privilege only on the container levels strictly *above* the level where you hold `MANAGE`, not at or below that level. On each of those higher levels, you can satisfy the requirement with `USE CATALOG` or `USE SCHEMA`, or with ownership or `MANAGE`. Because `MANAGE` on a container is inherited by all of its child objects, holding `MANAGE` at a higher level means you need no usage privileges below it. For example:

- `MANAGE` on a catalog requires no usage privileges. You can manage the catalog and any schema, table, or other object within it without `USE CATALOG` or `USE SCHEMA`.
- `MANAGE` on a schema requires `USE CATALOG` on the parent catalog (or ownership or `MANAGE` on that catalog), but not `USE SCHEMA` on the schema.
- `MANAGE` on a non-container object, such as a table, view, volume, or function, requires `USE CATALOG` on the parent catalog and `USE SCHEMA` on the parent schema (or ownership or `MANAGE` on either parent).

This reduction applies only to the metadata capabilities that `MANAGE`, and any child privileges of `MANAGE` such as `READ METADATA`, provide. Data-access privileges such as `SELECT` and `MODIFY` still require `USE CATALOG` and `USE SCHEMA`, even for users with `MANAGE`. For more information about what `MANAGE` allows, see [The `MANAGE` privilege](#).

The following table shows common operations and their required privileges:

Operation	Required privileges 
Read data from a table or view	USE CATALOG on catalog, USE SCHEMA on schema, SELECT on table or view
Write data to a table	USE CATALOG on catalog, USE SCHEMA on schema, MODIFY on table
Create a schema in a catalog	USE CATALOG on catalog, CREATE SCHEMA on catalog
Create a table in a schema	USE CATALOG on catalog, USE SCHEMA on schema, CREATE TABLE on schema (or catalog if granted at catalog-level)
Execute a function	USE CATALOG on catalog, USE SCHEMA on schema, EXECUTE on function
Read files from a volume	USE CATALOG on catalog, USE SCHEMA on schema, READ VOLUME on volume
Manage a catalog (grant privileges, transfer ownership, drop)	MANAGE on catalog (no usage privileges required)
Manage a schema (grant privileges, transfer ownership, drop)	USE CATALOG on catalog, MANAGE on schema
Manage a table, view, volume, or function (grant privileges, transfer ownership, drop)	USE CATALOG on catalog, USE SCHEMA on schema, MANAGE on the object

ALL PRIVILEGES behavior

`ALL PRIVILEGES` *implies* all applicable privileges for a specific object type, without Databricks explicitly granting each individual privilege. For example:

- `ALL PRIVILEGES` on a table implies the ability to perform `SELECT`, `MODIFY`, and `APPLY TAG`.
- `ALL PRIVILEGES` on a volume implies the ability to perform `READ VOLUME`, `WRITE VOLUME`, and `APPLY TAG`.
- `ALL PRIVILEGES` on a schema implies all schema-level privileges.
- `ALL PRIVILEGES` on a catalog implies all catalog-level privileges.

`ALL PRIVILEGES` does not include the `EXTERNAL USE SCHEMA`, `EXTERNAL USE LOCATION`, `MANAGE`, or `READ METADATA` privileges.

For more details on how `ALL PRIVILEGES` is evaluated and revoked, see [ALL PRIVILEGES](#).

The `MANAGE` privilege

The `MANAGE` privilege allows users to manage privileges on, transfer ownership of, and delete an object without being the owner. Having `MANAGE` is similar to ownership, but there are some important differences. See [Ownership versus the `MANAGE` privilege](#).

To exercise `MANAGE`, users need the appropriate usage privileges on the parent containers above the object where `MANAGE` is granted, but not on that object itself. For example, `MANAGE` on a catalog requires no usage privileges at all. For full details, see [Usage privilege requirements for `MANAGE`](#).

If `MANAGE` is granted on a container object, the user also gets `MANAGE` on all child objects.

For full details, see [MANAGE](#).

The `BROWSE` privilege

`BROWSE` allows users to discover objects and view their metadata without granting access to the underlying data. Users with `BROWSE` can see that an object exists, view its name, description, and tags, and request access to it without needing `USE CATALOG` or `USE SCHEMA`.

`BROWSE` is granted at the catalog level and applies to all objects within that catalog. Databricks recommends granting `BROWSE` on catalogs to the `All account users` group to make data discoverable throughout your organization.

For full details, see [BROWSE](#).

Composite and child privileges

Some privileges are related through a composite-child relationship. A *composite privilege* grants a broad set of capabilities, and each of its *child privileges* grants a subset of those capabilities. Child privileges let you delegate part of a composite privilege's access without granting the full privilege.

Composite and child privileges are granted and revoked independently. Granting a composite privilege does not grant its child privileges, and revoking a composite privilege does not revoke any child privilege that was granted explicitly.

The following are examples of composite and child privileges:

- `MANAGE` is a composite privilege, and `READ METADATA` and `MANAGE ACCESS CONTROL` are its children.
 - `READ METADATA` grants read-only visibility into an object's metadata, without the modification and management abilities that `MANAGE` includes. See [READ METADATA](#).
 - `MANAGE ACCESS CONTROL` grants the access-management subset of `MANAGE`, such as managing privileges, ownership, and [attribute-based access control \(ABAC\)](#) policies. It can be denied through an [ABAC DENY policy](#), but not granted.
- `MODIFY` is a composite privilege, and `INSERT`, `UPDATE`, and `DELETE` ([Beta](#)) are its children. Each grants a subset of the write access that `MODIFY` provides. See [Fine-grained DML privileges](#).

Ownership

Every securable object in Unity Catalog has an owner. The owner can be a user, service principal, or group. The principal that creates an object becomes its initial owner.

Ownership has a special denotation in Unity Catalog. Object owners can automatically perform all capabilities on the object they own. However, Databricks doesn't explicitly grant the `ALL PRIVILEGES` privilege to the owner. This means you won't see `ALL PRIVILEGES` returned when listing permissions using the Databricks API or with a `SHOW GRANTS` command.

Ownership doesn't inherit downward in Unity Catalog. However, object owners do automatically have the ability to manage all child objects. For example, if you own a catalog, you don't automatically own the child schemas within the catalog, but you can manage all child schemas. Similar to owners having all capabilities on their object without explicitly having `ALL PRIVILEGES`, Databricks also doesn't explicitly grant the `MANAGE` privilege in this case.

Object owners can perform other important operations on the object, including granting and revoking permissions, transferring ownership, and dropping the object.

NOTE

To avoid accidental data exfiltration, schema owners do not have the `EXTERNAL USE SCHEMA` privilege by default and external location owners do not have the `EXTERNAL USE LOCATION` privilege by default. See [Enable external data access to Unity Catalog](#).

To summarize, the owner of an object can do the following:

Capability	Description
All object capabilities	Owners automatically have all capabilities on the object they own. This is functionally the same as having <code>ALL PRIVILEGES</code> , but Databricks doesn't explicitly assign <code>ALL PRIVILEGES</code> to owners.
Manage child objects	Owners automatically get the ability to manage all child objects. This is functionally the same as having the <code>MANAGE</code> privilege on all child objects, but Databricks doesn't explicitly assign the <code>MANAGE</code> privilege to owners.
Grant and revoke permissions	Owners can grant and revoke privileges on the object to users and groups
Transfer ownership	Owners can transfer ownership of their object to another principal
Drop objects	Owners can delete the objects they own

For information about how to view and transfer ownership, see [Manage object ownership](#).

Ownership versus the `MANAGE` privilege

The `MANAGE` privilege grants a user the ability to grant and revoke privileges on the object. It does not grant the user all privileges on the object. However, users with `MANAGE` can explicitly grant themselves data access privileges such as `SELECT`.

In contrast, object owners have all capabilities on their object by default. Because ownership doesn't inherit downward to child objects, owners still require explicit grants on those child objects.

Usage privilege requirements are the same for both: owners and users with `MANAGE` need the appropriate usage privileges on the object's parent containers, but not on the object itself. See [Usage privilege requirements for `MANAGE`](#).

The following table summarizes the key differences between ownership and the `MANAGE` privilege:

Capability	Owner	<code>MANAGE</code> privilege 
All capabilities on the object	Yes, all capabilities are <i>implied</i> . This is different from explicitly having <code>ALL PRIVILEGES</code> . See Ownership .	No, privileges must be granted separately. Users with <code>MANAGE</code> can grant themselves these privileges.
Grant and revoke privileges on the object	Yes	Yes
Transfer ownership	Yes	Yes
Drop the object	Yes	Yes
Ability to manage child objects	Yes, manage capabilities on child objects are <i>implied</i> . See Ownership .	Yes, the <code>MANAGE</code> privilege is explicitly granted on child objects.
Requires usage privileges	No	Requires the appropriate usage privileges on the parent containers above the object where <code>MANAGE</code> is granted, but not on the object itself. <code>MANAGE</code> on a catalog requires no usage privileges.
 Ask Genie		

Capability	Owner	<code>MANAGE</code> privilege
Number of principals	Only one principal (user, service principal, or group)	Can be granted to multiple principals

To avoid accidental privilege escalation, the `ALL PRIVILEGES` privilege doesn't include the `MANAGE` privilege.

Privilege inheritance

NOTE

If you created your Unity Catalog metastore during the public preview (before August 25, 2022), you might be on an earlier privilege model that doesn't support the current inheritance model. You can upgrade to Privilege Model version 1.0 to get privilege inheritance. See [Upgrade to privilege inheritance](#).

Privilege inheritance is a key feature of the Unity Catalog permissions model. When you grant a privilege on a parent object, that privilege automatically applies to all current and future child objects. This simplifies access management by reducing the number of individual grants you need to make.

Privileges inherit downward through the object model hierarchy:

- A privilege granted on a catalog applies to all schemas in that catalog, and all tables, views, volumes, and functions in those schemas
- A privilege granted on a schema applies to all tables, views, volumes, and functions in that schema

For example, if you grant a user the `SELECT` privilege on a catalog, then that user can read all tables and views in that catalog (with the appropriate `USE CATALOG` and `USE SCHEMA` usage privileges).



Denials inherit downward the same way. When you deny a privilege on a parent object, that privilege is automatically denied on all current and future child objects.

These inheritance rules apply whether you assign a privilege manually with `GRANT` or use [attribute-based access control \(ABAC\)](#) policies to grant or deny it.

❗ IMPORTANT

Privileges granted on a metastore do not inherit to child objects. Metastore-level grants control metastore-scoped operations like `CREATE CATALOG` and `CREATE EXTERNAL LOCATION`, not access to data within the metastore.

The `READ METADATA` privilege is an exception. When granted on a metastore, `READ METADATA` inherits down to all objects in the metastore, and no usage privileges (`USE CATALOG` or `USE SCHEMA`) are required to exercise it. See [READ METADATA](#).

Ownership doesn't inherit downward in Unity Catalog. As the owner of an object, you're automatically granted all privileges on that object only. You don't automatically assume ownership of child objects created under your object. However, you do automatically get the `MANAGE` privilege on all new and existing child objects.

Inheritance examples

Suppose you grant `SELECT`, `USE CATALOG`, and `USE SCHEMA` at the catalog level to the `finance_team` group:

SQL

```
GRANT USE CATALOG, USE SCHEMA, SELECT ON CATALOG sales TO finance_team;
```

This grant allows the `finance_team` group to:



- Access the `sales` catalog
- Access all schemas in the catalog
- Read data from all current and future tables and views in the catalog

Suppose you grant `CREATE TABLE`, `USE CATALOG`, and `USE SCHEMA` at the catalog level to the `data_engineers` group:

SQL

```
GRANT USE CATALOG, USE SCHEMA, CREATE TABLE ON CATALOG analytics TO data_engineers;
```

This grant allows the `data_engineers` group to:

- Access the `analytics` catalog
- Access all schemas in the catalog
- Create tables in any current or future schema in the catalog

For instructions on how to grant and revoke privileges, see [Show, grant, and revoke privileges](#).

On this page

Was this page helpful?



Yes



No



Send feedback